

ASSIGNMENT-1 CO2

1. HEADER

Name:	B Baby Suharshitha
Roll No.:	05
Course Code & CO Reference:	CSA0920, CO2
Assignment Set & Question No.:	Direct-Descriptive, Q05
GitHub Repository Link:	https://github.com/suharshithabathina/CSA0920-JAVA-
Submission Date:	July 31, 2026

Assigned Question Text:

Reversing a String with StringBuilder:

Write a Java program that accepts a string from the user and reverses it using the StringBuilder class's built-in reverse() method, rather than manually looping through the characters. Display both the original string and the reversed result.

Concepts: StringBuilder, reverse().

2. PROBLEM UNDERSTANDING

The objective of this program is to process user-provided textual input and compute its character-reversed counterpart without performing explicit loop-based character swapping. It tests the understanding of Java's mutable

sequence class (StringBuilder) and its optimized, standard-library API call `reverse()` for high-performance string manipulation.

3. APPROACH / DESIGN NOTES

- **Class Selection:** Used `java.util.Scanner` for taking standard keyboard input from the user.
- **String Manipulation Choice:** Selected `java.lang.StringBuilder` rather than standard `String` concatenation or manual character array inversion. Standard Java `String` objects are immutable; manipulating them via manual loops creates multiple temporary garbage objects. `StringBuilder` provides an in-place mutable character sequence where `reverse()` operates efficiently in $O(n)$ time complexity.
- **Rejected Alternative:** Considered converting the string to a `char[]` array and using a two-pointer swapping technique in a for loop. This approach was rejected as the assignment specifically evaluates standard library API proficiency using `StringBuilder.reverse()`.

4. SOURCE CODE

```
import java.util.Scanner;

public class StringReverser {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a string to reverse: ");

        String originalString = scanner.nextLine();

        StringBuilder sb = new StringBuilder(originalString);
```

```
sb.reverse();

String reversedString = sb.toString();

System.out.println("Original String : " + originalString);

System.out.println("Reversed String : " + reversedString);

scanner.close();

}

}
```



Online Java Compiler

Programiz PRO >

StringReverser.java



Run

```
1- import java.util.Scanner;
2- public class StringReverser {
3-     public static void main(String[] args) {
4-         Scanner scanner = new Scanner(System.in);
5-         System.out.print("Enter a string to reverse: ");
6-         String originalString = scanner.nextLine();
7-         StringBuilder sb = new StringBuilder(originalString);
8-         sb.reverse();
9-         String reversedString = sb.toString();
10-        System.out.println("Original String : " + originalString);
11-        System.out.println("Reversed String : " + reversedString);
12-        scanner.close();
13-    }
14- }
```

Output

Clear

Enter a string to reverse: Suharshitha
Original String : Suharshitha
Reversed String : ahtihsrathS

=== Code Execution Successful ===

JS

5. TEST CASES

s.no.	Input	Expected Output	Actual Output
1	Suharshitha (Typical case)	Original String : Suharshitha Reversed String : ahtihsrahuS	Original String : Suharshitha Reversed String : ahtihsrahuS
2	Racecar (Edge Case - Palindrome)	Original: racecar Reversed: racecar	Original: racecar Reversed: racecar
3	(Boundary Case - Empty String)	Original String : Reversed String :	Original String : Reversed String :

6. SCREENSHOTS OF EXECUTION

Test Case 1 (Typical case)

```
Enter a string to reverse: Suharshitha
Original String : Suharshitha
Reversed String : ahtihsrahuS
```

Test Case 2 (Edge Case - Palindrome)

```
Enter a string to reverse: racecar
Original String : racecar
Reversed String : racecar
```

Test Case 3 (Boundary Case - Empty String)

```
Enter a string to reverse:
Original String :
Reversed String :
```

7. REFLECTION

Through this assignment, I reinforced my understanding of memory mutability in Java, specifically how `StringBuilder` avoids the memory overhead of creating multiple temporary string instances during manipulation. I ensured that empty string boundary conditions were handled gracefully by the standard library without throwing a `NullPointerException` or `IndexOutOfBoundsException`.

8. DECLARATION

"This is my own work. Any AI/external tool assistance used is disclosed above. No external assistance was used in writing the core code logic."